

Software Tools Lecture 2

The shell, shell scripting, and a bunch of silly commands that you just need to know

Gretchen Hallett

Progression

Last time, we talked about how to connect to a computer and run basic commands

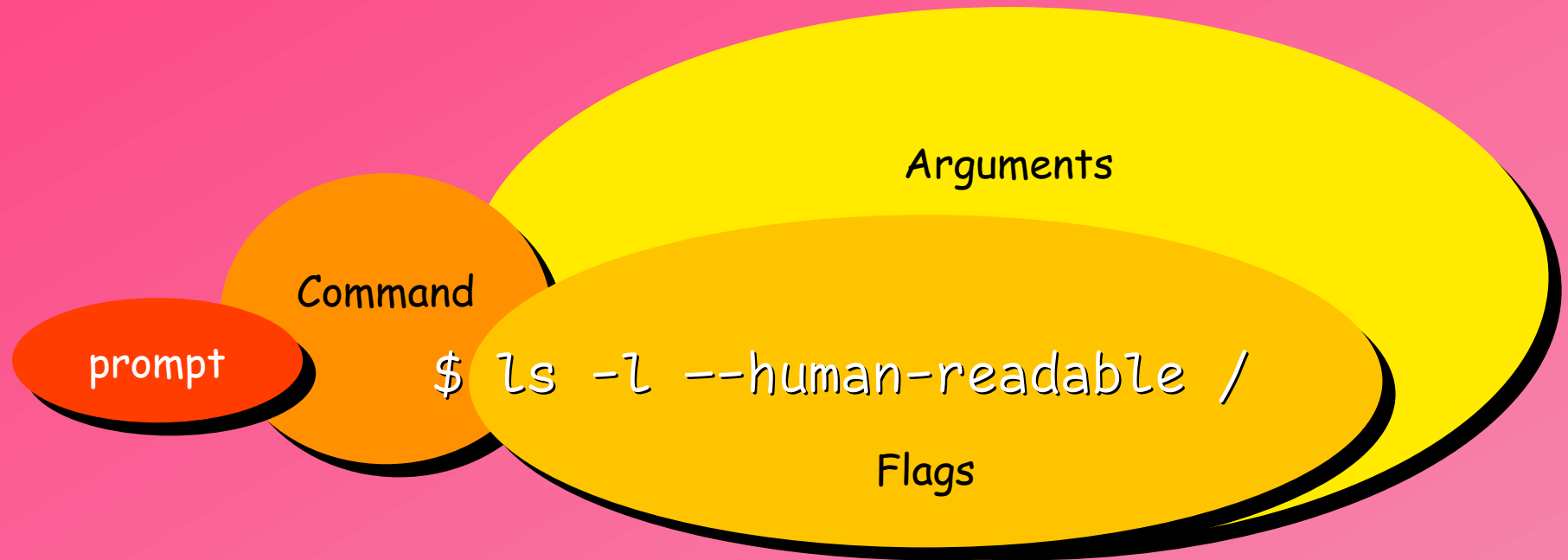
*This week, its how we use the command line
And how to automate it*

- Learning the shell will take time and practice
- Its not enough to *understand the theory*
 - You have to use it

If you don't you'll still struggle in 4th year/work

If you switch to it **entirely** for the next month at least...

- You'll be really slow at first, but it will get easier and easier and you won't look back
- ...and the exam will be easy



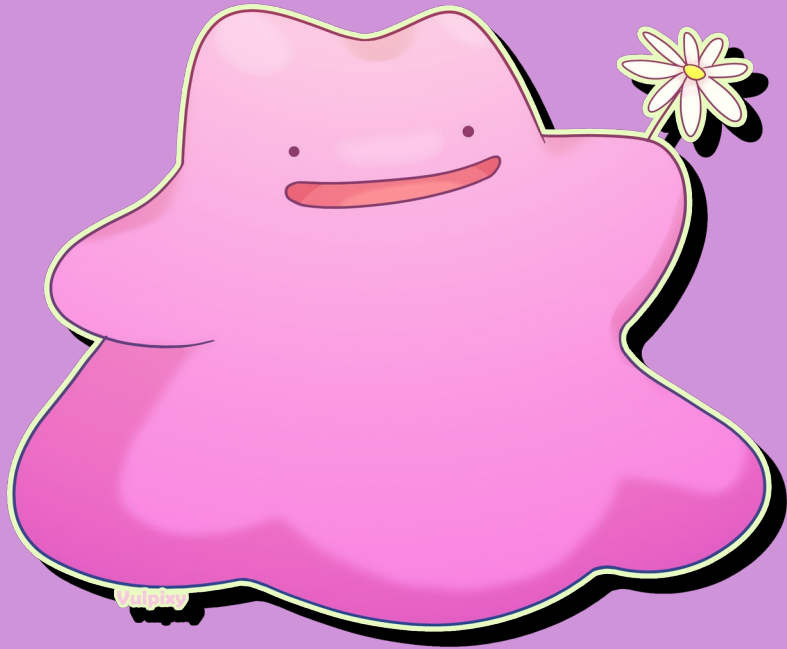
I think of commands in three groups

Those for getting information

Those for filtering information

Those for doing stuff

(Some can do all three)



I want to find all the markdown files
in my current folder (or the
immediate subfolders within it)

I want to see how many mention
Joseph

```
$ ls ./**  
./README.md          ./permissions.pdf  ./pipes_1.pdf  
./pipes_2.pdf        ./scripting_1.pdf  
./scripting_2.pdf    ./shell.pdf        ./slides.md  
  
./lab:  
permissions.md pipes.md          script.md  
shell.md
```

```
$ ls ./** | tr '\t' ' '
./README.md
./permissions.pdf
./pipes_1.pdf
./pipes_2.pdf
./scripting_1.pdf
./scripting_2.pdf
./shell.pdf
./slides.md

./lab:
permissions.md
pipes.md
script.md
shell.md
```

Pipes!

pipes are one of the most powerful features of the shell

| takes the output of whatever is on the left hand side and pipes it in as input to the right hand side

> takes output from the left, and creates a new file in the path on the right; and >> which appends the 0.2 instead of overwriting

< which takes the contents of the path on the right and feeds it into the output of the command on the left

```
$ ls /** | tr '\t' '\n'
```

- ./README.md
- ./permissions.pdf
- ./pipes_1.pdf
- ./pipes_2.pdf
- ./scripting_1.pdf
- ./scripting_2.pdf
- ./shell.pdf
- ./slides.md

- ./lab:
 - permissions.md
 - pipes.md
 - script.md
 - shell.md

The `tr` command *transliterates* characters from its first argument and replaces them with characters from its second

So take the tabulated output of `ls`, and replace tabs (`\t`) with newlines (`\n`)!

I just want the files ending in `.md` though... so lets filter again

```
$ find . -name \*.md  
./README.md  
./lab/permissions.md  
./lab/pipes.md  
./lab/script.md  
./lab/shell.md  
./slides.md
```

There is almost always
more than one way of
doing things

```
$ for f in $(find . -name \*.md);  
do  
    echo "${f}";  
done  
./README.md  
./lab/permissions.md  
./lab/pipes.md  
./lab/script.md  
./lab/shell.md  
./slides.md
```

Iterate through
things!

Sometimes we want to use the output of a shell command to build the next one

Using `$()` or ``` (old notation) takes the command inside, runs it and replaces itself on the prompt with the output

For example `sudo $(!!)` runs the last command `(!!)` as root (useful when you forget `sudo`)

There is also `<()` which replaces the output of the command with a temporary file containing the output

Shells have loops and if statements, but the syntax is funky and they're treated as *builtins* not commands

```
for var in 1 2 3; do echo "${var}"; done  
if test ${x} -gt 3; then echo big x; fi
```

You can create variables in a shell

`x=5` (no spaces round =) sets `x` to the string "5"

`${x}` gets the value of `x` and inserts it into the string (always wrap in "" unless you know what your doing)

`files=$(ls *.md)` makes `${files}` be the output of `ls`

Most shells will let you do magic with variables

`${x:?x is not defined}` will throw an error if `x` isn't set for example.

```
$ for f in $(find . -name \*.md);  
do  
    matches=$(grep -iH joseph "${f}");  
    printf "%d match" $(wc -l <<<${matches})  
    echo ${matches}  
done  
1 matches  
./README.md:Joseph smells!
```

Only one!
I could change it by hand... but I am
very very lazy

```
$ for f in $(find . -name \*.md);  
do  
    sed -ie 's/Joseph/Gretchen/g' \  
        $(grep -iH joseph "${f}" | cut -d: -f1)  
done
```

Use the stream editors to make the
edit in place

```
$ find . -name \*.md -exec sed  
-ie 's/Joseph/Gretchen/g' {} \  
+
```

I could reduce this further to a
oneliner but this isn't the only unit
I teach...

And it is still too long

```
$ transition() {  
  find . -name \*.md \  
    -exec sed -ie 's/Joseph/Gretchen/g' {} \+;  
}
```

So lets make it a built in
function... then we can just call
that

But this will disappear when I
start a new shell...

If I want the function to stick around I *could* stick it in my `~/.profile` file. The shell reads this every time it starts up and would recreate the function each time

This is great when I want to override default behaviour (`cp() { /bin/cp -a $@; }`) for example but for complex scripts gets a bit unweildy

(or `~/.bashrc` or `~/.bash_profile` if I use bash (most Linux); or `~/.zshrc` if I use zsh (Macs and some Linux) or `~/.kshrc` on OpenBSD but I use Bash and just stick `.` `~/.profile` in all those files and stick to the POSIX stuff mostly... which uses `~/.profile`)

What happens when a program loads?

Linux supports *many* different program formats... but the most common are ELF (a compiled binary) and shellscript...

When an ELF file is exec'd it is passed to the dynamic linker (/usr/lib/ld-linux.so.2) to have the appropriate libraries loaded...

But if the file is a text file and starts with #! then something special happens...

```
#!/usr/bin/env bash
# This is a shell script. Everything in it will be sent line by
# line to the program listed after the #! (pronounced shèbang)

# If I wanted the script to be interpreted by Python I could
# write #! /usr/local/bin/python3

# But because every system puts the programs in a slightly
# different place... better to use /usr/bin/env python3 which will
# find the right place for the system you're on... yay portability!

echo "The program's name is ${0} and the arguments passed are ${@}!"

if [[ ! -z "${masochist}" ]]; then
    echo "...try and find a #! line for a C file"
    echo "that compiles and runs it as a program ;-)"
fi
```

With the script written we just need to do two more things... normally text files aren't executable; so you can't run them

```
$ chmod +x myscript
```

We can then run it by passing the shell the full path to it `${PWD}/myscript` (or wherever you saved it) but its easier if we install it onto the `$ {PATH}`...

The `${PATH}` variable is where the system will look for scripts to run. First matching program on the path wins. You are free to update it.

```
$ echo "${PATH}"
```

```
/usr/share/Modules/bin:/usr/local/  
cuda-11.4/bin:/usr/local/bin:/usr/bin:/  
usr/local/sbin:/usr/sbin:/opt/puppetlabs/  
bin:/var/lib/snapd/snap/bin:/opt/software/  
bin:/home/jh18636/perl5/bin
```

Adding `export PATH="${PATH} ~/.local/bin/"` to your `.profile` is usually sensible, then you can copy your scripts there and they should just work.

Adding . Or prepending your scripts folder to the path can lead to trolling/vulnerabilities. If you need to modify a system program use `~/.profile` instead

Within a shell script...
normal programming
rules apply...

But it is like
programming in a shell...
and you **might** want to
turn off some of the
hacky stuff...

If a shell command
fails... we normally just
run the next one... but
in a program we might
want to trigger an
error...

```
set -e # Exit on first  
error  
set -o pipefail # If a  
command in a pipe  
fails... then stop
```

More exist! They're
useful... but append key
commands with
a_random_command_that_
fails || (1>&2 echo
"This has all gone
wrong!")
So you can tell when
things break!

Other useful shell stuff!

- If you suffix a command with a & it runs in the background
- If you start a command and hit Control-Z it is suspended (stopped and you're returned to the shell)
 - Run fg to start it again, bg to background it, or jobs to see suspended things

Other useful shell stuff!

- If a program is waiting for input... you can say no more input coming with Control-D
- If a running program needs to be killed quickly hit Control-C
- If you want to see what a program is doing hit Control-T (but few things support it)

WE COULD JUST GO THROUGH
A SUPER LONG LIST FROM
POSIX OF EVERY COMMAND
LINE TOOL YOU MIGHT EVER
WANT TO USE BUT THAT
SOUNDS BORING, DULL AND
TEDIOUS, ESPECIALLY FOR ME...

awk :: useful tool for dealing with files	env :: inspect environment variables	perl :: better (?) shellscript	tail :: get the bottom
basename :: just the filename	file :: what are you looking it?	ps :: what's running?	tar :: archive stuff
bc :: maths stuff	finger :: what are other people doing	read :: input	tee :: write stuff to a file and look at it
cat :: join files	gz :: compress stuff	rm :: delete stuff	trap :: signals
comm :: find common lines	head :: get the top of a file	screen :: suspend terminal sessions and have multiple sessions in a single terminal (and connect to a serial console) (see also tmux)	uniq :: remove duplicates
cp :: copy files	install :: like cp with permissions	sendmail :: send email on a commandline (you'll need a mailserver though)	wait :: wait for backgrounded processes
crontab :: run commands at a time	kill :: kill a process	source :: read another file	wall :: terrible chat
cut :: extract fields from lines	less/more :: view a long file	sed :: editing pipes	wc :: count lines, words or characters
dd :: bulk write to disks/tapes	ln :: alias files and folder	sort :: sorts	xargs :: helps avoid looping
df :: find free space	man :: THE MANUAL	strings :: get things that look like text from a file	yes :: helps avoid typing yes a lot
diff :: find differences	mkdir :: make a directory	stty :: configure your terminal	(also these logic ops)
du :: how much space is this using	mv :: rename stuff		&& :: if command on left succeeds run command on right
ed :: the standard text editor... (for use when you don't have a screen)	patch :: automated changing of stuff		 :: if command on left fails run command on right

But maybe lets just try writing things and giving it a go?

Oh but all those utilities on the last slide are still examinable...

Read the man pages ;-)
(Yes I am evil)

What is the output of this shell pipeline?

```
$ echo Hello $(echo world | tr a-z A-Z | sort)
```



Red:Hello WORLD

Green:HELLO WORLD

Yellow:Hello DLORW

Cyan:HELLO dlOrw

????

????

????

????

What is the output of this shell pipeline, when run as root?

```
$ echo Never run 'rm -rf /'
```



Red: Never run 'rm -rf /'

Green: Never run rm -rf /

Yellow: Nothing: it erases your computer

Cyan: Never run "rm -rf /"

????

????

????

????